arXiv:2404.15255v1 [cs.LG] 23 Apr 2024

How to use and interpret activation patching

Stefan Heimersheim
stefan.heimersheim@gmail.com

Neel Nanda

Abstract

Activation patching is a popular mechanistic interpretability technique, but has many subtleties regarding how it is applied and how one may interpret the results. We provide a summary of advice and best practices, based on our experience using this technique in practice. We include an overview of the different ways to apply activation patching and a discussion on how to interpret the results. We focus on what evidence patching experiments provide about circuits, and on the choice of metric and associated pitfalls.

1 Introduction

1.1 What is activation patching?

Activation patching (also referred to as Interchange Intervention, Causal Tracing, Resample Ablation, or Causal Mediation Analysis) is the technique of replacing internal activations of a neural net. It is also known as Causal Tracing, Resample Ablation, Interchange Intervention or more generally as Causal Mediation Analysis. Variants of this technique have been widely used in the literature (Vig et al., 2020; Geiger et al., 2021a; Geiger, Richardson, and Potts, 2020; Soulos et al., 2019; Finlayson et al., 2021; Geiger et al., 2021b; Meng et al., 2022; Wang et al., 2022; Chan et al., 2022; Hase et al., 2023; Hanna, Liu, and Variengien, 2023; Conmy et al., 2023; Todd et al., 2023; Hendel, Geva, and Globerson, 2023; Feng and Steinhardt, 2023; Lieberum et al., 2023; Cunningham et al., 2023; Stolfo, Belinkov, and Sachan, 2023; Goldowsky-Dill et al., 2023; McDougall et al., 2023; Geva et al., 2023; Huang et al., 2023; Merullo, Eickhoff, and Pavlick, 2023; Tigges et al., 2023). Here we focus on the technique where we overwrite some activations during a model run with cached activations from a previous run (on a different input), and observe how this affects the model's output.

1.2 How is this related to ablation?

Ablation is the common technique of zeroing out activations. Activation patching is more targeted and controlled: We replace activations with other activations rather than zeroing them out. This allows us to make targeted manipulations to locate specific model behaviours and circuits.

1.3 An example

For example, let's say we want to know which model internals are responsible for factual recall in ROME (Meng et al., 2022). How does the model complete the prompt "The Colosseum is in" with the answer "Rome"? To answer this question we want to manipulate the model's activations. But the model activations contain many bits of information: This is an English sentence; The landmark in question is the Colosseum; This is a factual statement about a location.

Ablating some activations will affect the model if these activations are relevant for *any* of these bits. But activation patching allows us to *choose* which bit to change and *control* for the others. Patching with activations from "Il Colosseo è dentro" locates where the model stores the language of the prompt but may use the same factual recall machinery. Patching with activations from "The Louvre is in" locates which part of the model deals with the landmark and information recall. Patching between

"The Colosseum is in the city of" and "The Colosseum is in the country of" locates the part of the model that determines *which* attributes of an entity are recalled.

A simple activation patching procedure typically looks like this:

1. Choose two similar prompts that differ in some key fact or otherwise elicit different model behaviour:

   E.g. "The Colosseum is in" and "The Louvre is in" to vary the landmark but control for everything else.

2. Choose which model activations to patch

   E.g. MLP outputs

3. Run the model with the first prompt—the source prompt—and save its internal activations

   E.g. "The Louvre is in" (source)

4. Run the model with the second prompt—the destination prompt—but overwrite the selected internal activations with the previously saved ones (patching)

   E.g. "The Colosseum is in" (destination)

5. See how the model output has changed. The outputs of this patched run are typically somewhere between what the model would output for the un-patched first prompt or second prompt

   E.g. observe change in the output logits for "Paris" and "Rome"

6. Repeat for all activations of interest

   E.g. sweep to test all MLP layers

1.4 What is this document about

We want to communicate useful practical advice for activation patching, and warn of common pitfalls to avoid. We focus on three areas in particular:

1. What kind of patching experiments provide which evidence? (Section 2)
2. How should you interpret activation patching results? (Section 3)
3. What metrics you can use, what are common pitfalls? (Section 4)

For a general introduction to mechanistic interpretability in general, and activation patching in particular we refer to ARENA[1] chapter 1 (in particular activation patching in chapter 1.3) as well as the corresponding glossary entries on Neel Nanda's website[2].

2 What kind of patching experiments should you run?

2.1 Exploratory and confirmatory experiments

In practice we tend to find ourselves in one of two different modes of operation: In exploratory mode we run experiments to find circuits and generate hypotheses. In confirmatory mode we want to verify the circuit we found and check if our hypothesis about its function is correct.

In *exploratory* patching we typically patch components one at a time, often in a sweep over the model (layers, positions, model components). We do this to get an idea of which parts of a model are involved in the task in question, and may be part of the corresponding circuit.

In *confirmatory* patching we want to confirm a hypothesised circuit by verifying that it actually covers all model components needed to perform the task in question. We typically do this by patching many model components at once and checking whether the task performance behaves as expected. A well-known example of patching for circuit verification is Causal Scrubbing (Chan et al., 2022).

[1] ARENA: `https://arena3-chapter1-transformer-interp.streamlit.app/`

[2] `https://neelnanda.io/glossary`

2.2 Which components should you patch

Patching can be done on different levels of granularity determining the components to patch. For example, we may patch the residual stream at a certain *layer* and *position*, or the output of a certain MLP [*layer*, *position*] or Attention Head [*layer*, *head*, *position*]. At even higher granularity we could patch individual neurons or sparse autoencoder features.

An even more specific type of patching is path patching. Usually, patching any component will affect *all* model components in later layers ("downstream"). In path patching instead we let each patch affect only a single target component. We call this patching the "path" between two components. For details on patch patching we refer to ARENA section 1.3.4.

Path Patching can be used to understand whether circuit components affect each other directly, or via mediation by another component. For example if we want to distinguish between *mediation* (component A affects output C via component B), and *amplification/calibration* (component A affects output C directly, but component B reads from A and also affects output C by boosting or cancelling the signal to amplify or calibrate component A). These two structures look identical in default component patching, but different in path patching: a direct connection (composition) between A and C exists only in the second case.

As a rule of thumb, you want to start with low-granularity patching (e.g. residual stream patching), then increase granularity, and finally use path patching to test which components interact with each other. Fast approximations to activation patching, such as attribution patching (see Nanda, 2023, and also AtP*, **atpstar**) can help speed up this process in large models.

2.3 Noising and Denoising

There are multiple ways to do activation patching. The techniques differ in what the source (source of activations / model run from which the activations are copied) and destination prompt (destination that is overwritten / model run in which the activations will be inserted, this is called base in Interchange Interventions language, Geiger et al., 2021b) are. *The use of words "source" and "destination" is unrelated to their meaning in Transformer attention.*

The two main methods are Denoising and Noising (see the next section for other methods).

1. **Denoising**: We can patch activations from a clean first prompt into a corrupted second prompt "clean → corrupt". That is running the model on the clean prompt while saving its activations, then running the model on the corrupted prompt while overwriting some of its activations with previously saved clean-prompt activations. We observe which patch restores the clean-prompt behaviour, i.e. patching which activations were **sufficient to restore** the behaviour.
2. **Noising**: Or you can patch activations from a corrupted first prompt into a clean second prompt "corrupt → clean". That is running the model on the corrupted prompt while saving its activations, then running the model on the clean prompt while overwriting some of its activations with previously saved corrupt-prompt activations. We observe which patch breaks the clean-prompt behaviour, i.e. patching which activations were **necessary to maintain** for the behaviour.

An important and underrated point is that these two directions can be very different, and are not just symmetric mirrors of each other. In some situations denoising is the right tool, and in others it's noising, and understanding the differences is a crucial step in using patching correctly.

Technique	Source (saved)	Source run input	Destination / Base (overwritten)	Destination / Base run input	Observation
Clean → corrupted (Denoising, Causal Tracing[2])	First run activations (clean)	Clean tokens	Second run activations (corrupted)	Corrupt tokens	What restores behaviour
Corrupted → clean (Noising, Resample Ablation)	First run activations (corrupted)	Corrupt tokens	Second run activations (clean)	Clean tokens	What breaks behaviour

For now we round patching effects to "if I patch these activations the model performance is / isn't affected". We discuss metrics and measuring patching effects in the last section.